



Chapitre 3 : Traitement de données en table

Leçon 1 : Indexation de tables et importation de données depuis un fichier texte tabulé ou CSV

Thème : Traitement de données en table

Enseignant : Housseem Lahiani

1. Introduction

L'indexation de tables et l'importation de données depuis des fichiers texte tabulés ou CSV sont des compétences essentielles dans le traitement des données. Dans ce cours, nous allons explorer ces concepts et apprendre comment manipuler des tables en utilisant des tableaux doublement indexés ou des tableaux de p-uplets qui partagent les mêmes descripteurs.

2. Indexation de tables

2.1. Définition

L'indexation de tables est une méthode qui vise à organiser et structurer les données tabulaires. Elle a pour objectif de faciliter l'accès, la recherche et la manipulation des données en utilisant des index spécifiques.

2.2. Tableaux doublement indexés

Dans un tableau de données les deux dimensions sont les lignes et les colonnes. Chaque ligne représente un enregistrement distinct, et chaque colonne représente un descripteur spécifique. Les valeurs dans le tableau sont organisées en fonction de ces deux dimensions.

Exemple :

ID	Nom	Age	Ville
1	Alice	25	Paris
2	Bob	30	New York
3	Claire	28	Sydney

Dans cet exemple, on peut utiliser l'indexation double sur les descripteurs "ID" et "Nom" pour accéder rapidement aux données d'une ligne spécifique.

Chaque ligne de ce tableau représente une entrée distincte avec des valeurs pour chaque colonne spécifique (ID, Nom, Age, Ville). Ce format tabulé est communément utilisé pour stocker et présenter des données dans des fichiers texte tabulés, des feuilles de calcul, ou des bases de données relationnelles.

2.3. Tableaux de p-uplets

Les tableaux de p-uplets sont des structures de données qui regroupent les données d'une table en utilisant des p-uplets (tuples) qui partagent les mêmes descripteurs. Chaque p-uplet représente une ligne de la table.

Exemple :

Alice	25	Paris
Bob	30	New York
Claire	28	Sydney

Dans cet exemple, chaque p-uplet représente une personne avec ses attributs (nom, âge, ville). Les descripteurs sont partagés entre tous les p-uplets.

3. La structure `with` en Python :

En Python, la structure `with` est utilisée pour créer un contexte de gestion de ressources, notamment lors de l'ouverture et de la fermeture de fichiers. Elle assure une gestion propre et sécurisée des ressources en garantissant que les ressources sont correctement libérées, même en cas d'erreurs pendant l'exécution du code.

Exemple d'utilisation avec l'ouverture de fichiers :

```
# Utilisation de with pour ouvrir un fichier

with open("exemple.txt", "r") as fichier:

    # Code à l'intérieur du bloc with

    lignes = fichier.readlines()

    # Pas besoin d'appeler fichier.close() ici

# À ce stade, le fichier est automatiquement fermé, même si une
# exception a été levée
```

Dans cet exemple, le bloc `with` ouvre le fichier "exemple.txt" en mode lecture. Le code à l'intérieur du bloc `with` s'exécute avec le fichier ouvert, et une fois que le bloc est quitté, le fichier est automatiquement fermé. Cela rend le code plus lisible, évite les oublis de fermeture du fichier, et garantit une gestion correcte des ressources.

Cependant si on n'utilise pas de `with` le code doit être comme suit :

```
# Ouverture du fichier sans with

fichier = open("exemple.txt", "r")

# Code à l'intérieur du bloc sans with

lignes = fichier.readlines()

# Fermeture explicite du fichier après avoir terminé

fichier.close()
```

En résumé, `with` simplifie la gestion des ressources en automatisant les tâches de démarrage et d'arrêt, ce qui est particulièrement utile lors de l'interaction avec des fichiers, des bases de données ou d'autres ressources nécessitant une fermeture explicite.

4. Importation de tables depuis un fichier texte tabulé ou CSV

4.1. Fichiers texte tabulés

Les fichiers texte tabulés sont des fichiers où les données sont organisées en table avec des séparateurs spécifiques pour les colonnes. Ils peuvent être importés dans un programme de traitement des données pour être manipulés. Dans l'exemple ci-dessous on peut dire que les

données sont formatées de manière tabulée, car les valeurs de chaque colonne sont séparées par des tabulations. Les fichiers texte tabulés sont des fichiers dans lesquels les données sont organisées en lignes et colonnes, et la séparation entre les colonnes est souvent réalisée à l'aide de caractères spécifiques, tels que des tabulations.

Les valeurs des colonnes (Nom, Age, Ville) sont alignées verticalement et séparées par des tabulations, créant ainsi une structure tabulée. Ce format est couramment utilisé pour stocker des données tabulaires dans des fichiers texte, et il est souvent compatible avec des logiciels de traitement de texte, des tableurs ou des bases de données qui peuvent interpréter ces tabulations pour afficher ou importer les données de manière organisée.

Exemple de fichier texte tabulé :

```
Nom  Age  Ville
Alice 25  Paris
Bob   30  New York
Claire 28  Sydney
```

Si nous mettons ses données tabulées dans un fichier txt en le nommant exemple.txt et nous voulons par la suite importer ces données avec python on doit utiliser ce bout de code :

```
# Lecture du fichier texte
with open("exemple.txt", "r") as fichier:
    lignes = fichier.readlines()

# Affichage des lignes lues
for ligne in lignes:
    print(ligne.strip())      # Utilisation de strip() pour
                              retirer les caractères de nouvelle ligne
```

4.2. Fichiers CSV

Les fichiers CSV (Comma-Separated Values) sont des fichiers texte où les données sont représentées sous forme de valeurs séparées par des virgules (ou des points-virgules). Ils sont couramment utilisés pour stocker des données tabulaires.

Exemple de fichier CSV :

```
Nom,Age,Ville
Alice,25,Paris
Bob,30,New York
Claire,28,Sydney
```

Pour la suite de notre travail, vous devez mettre ce contenu au-dessus dans un fichier txt, il faut changer l'extension du fichier à .csv par la suite de manière à ce que le nom de fichier devient donnees.csv

4.3. Importation de données avec Python

Python offre des bibliothèques et des modules pour importer et manipuler des tables depuis des fichiers texte tabulés ou CSV.

Il existe deux principales façons d'importer des données CSV en Python : en utilisant le module `csv` de la bibliothèque standard et en utilisant la bibliothèque externe `pandas`. Chacune de ces méthodes offre des fonctionnalités spécifiques pour travailler avec des données tabulaires. (Pour utiliser la bibliothèque pandas, vous devez l'installer au préalable en utilisant la commande `pip install pandas`.)

a. Importation avec le module `csv` :

Le module `csv` de la bibliothèque standard de Python offre une approche simple et directe pour lire et écrire des fichiers CSV. Voici un exemple :

```
import csv

# Lecture des données depuis un fichier CSV
with open("donnees.csv", "r") as fichier_csv:
    reader = csv.reader(fichier_csv)
    lignes = list(reader)

# Affichage des lignes lues
for ligne in lignes:
    print(ligne)
```

Ce code utilise `csv.reader` pour lire les données ligne par ligne depuis un fichier CSV. `csv.reader` est utilisé pour lire les données du fichier CSV. Les données sont ensuite converties en une liste de listes (lignes), où chaque sous-liste représente une ligne du fichier CSV.

Pour transformer les données d'un fichier CSV en dictionnaire, vous pouvez utiliser la classe `csv.DictReader` fournie par le module `csv`. Voici comment vous pourriez le faire :

```
import csv

# Lecture des données depuis un fichier CSV en tant que
# dictionnaire
with open("donnees.csv", "r") as fichier_csv:
    # Création d'un objet DictReader pour lire le fichier CSV
```

```
reader = csv.DictReader(fichier_csv)

# Parcours des lignes du fichier CSV
for ligne in reader:
    # Chaque ligne est un dictionnaire
    print(ligne)
```

Avec `csv.DictReader`, chaque ligne du fichier CSV est représentée sous la forme d'un dictionnaire, où les clés du dictionnaire sont les noms des colonnes. Par exemple, si le fichier CSV a une en-tête avec des colonnes "Nom", "Age" et "Ville", chaque ligne sera un dictionnaire avec ces clés.

Cela simplifie l'accès aux données en utilisant les noms de colonnes comme clés dans les dictionnaires. Vous pouvez ensuite manipuler ces données de manière plus intuitive.

b. Importation avec la bibliothèque `pandas` :

La bibliothèque `pandas` est une puissante bibliothèque pour l'analyse de données en Python, et elle simplifie considérablement la manipulation de données tabulaires. Voici un exemple :

```
import pandas as pd

# Lecture des données depuis un fichier CSV
donnees = pd.read_csv("donnees.csv")

# Affichage des données
print(donnees)
```

Avec `pandas`, vous pouvez charger les données directement dans une structure de données appelée DataFrame, ce qui facilite l'exploration et la manipulation des données.

En résumé, le module `csv` est utile pour une manipulation basique de fichiers CSV, tandis que `pandas` offre une solution plus avancée et puissante pour l'analyse de données tabulaires. Le choix entre les deux dépend de la complexité de vos besoins et du niveau de fonctionnalités que vous recherchez.

Voici un exemple d'utilisation de la bibliothèque pandas pour importer un fichier CSV :

```
import pandas as pd  
table = pd.read_csv("donnees.csv")  
print(table)
```

Dans cet exemple, le fichier "donnees.csv" est importé en utilisant la fonction `read_csv()` de la bibliothèque `pandas`, et la table est stockée dans la variable `table`.

Conclusion

Dans ce cours, nous avons exploré l'indexation de tables et l'importation de données depuis des fichiers texte tabulés ou CSV. Nous avons appris à manipuler des tables en utilisant des tableaux doublement indexés ou des tableaux de p-uplets. Ces compétences sont essentielles pour le traitement des données et peuvent être utilisées dans de nombreux domaines tels que l'analyse de données, la science des données et la gestion de bases de données.